UFR de mathématique et d'informatique
Université de Strasbourg

Intelligence Artificielle

Licence 3 Informatique

Évaluer la contribution des attributs d'entrée dans un réseau de neurones artificiels

---

Ahmet Berk ATES, Antoine GAUTHERON,
Baptiste GAUCHARD

Année universitaire 2024/2025

Table des matières

1 Introduction **2**

2 Méthodologie **2**
2.1 Construction du réseau de neurones artificiels 2
2.1.1 Dataset et construction 2
2.1.2 Architecture et entrainement 3
2.2 Sélection d'instances 3
2.3 Interprétation du modèle 4
2.4 Examen de la contribution des attributs 6

3 Résultats **6**
3.1 Capacités de prédiction du modèle 6
3.2 Instances choisies 10
3.3 Comportement local du modèle 10
3.4 Contribution des attributs 11

4 Discussion **14**

5 Conclusion **15**

Annexe A Sélection d'instances **16**

1 Introduction

Avec l'essor des réseaux de neurones artificiels comme outil de décision informatique, il devient essentiel de comprendre les mécanismes impliqués dans leur fonctionnement. Malgré leur popularité dans de nombreux domaines, le processus décisionnel des réseaux de neurones artificiels peut manquer de transparence. Dans ce contexte, nous nous intéresserons particulièrement à leur comportement au niveau local et nous évaluerons la contribution des attributs d'entrée pour des instances difficiles à prédire. Nous pourrons ainsi interpréter au mieux le processus décisionnel général du modèle. Notre rapport présentera le cheminement de notre travail en exposant nos différentes réflexions et réalisations tout en explicitant leurs justifications et leurs résultats.

2 Méthodologie

2.1 Construction du réseau de neurones artificiels

2.1.1 Dataset et construction

Pour débuter notre expérimentation, nous avons d'abord implémenté un modèle de réseau de neurones en complétant le squelette fourni dans le fichier *NeuralNet.py*, selon le principe de l'apprentissage supervisé.

Ce modèle inclut une particularité : une fonction d'évolution du learning rate au fil des époques. L'objectif initial était de le faire décroître progressivement afin d'affiner l'apprentissage. Cependant, comme illustré ci-dessous, la formule que nous avons utilisée provoque par erreur une augmentation du learning rate au cours du temps, en raison d'une inversion dans l'expression.

```
self.learning_rate * (epoch/self.n_epoch)
```

Nous verrons par la suite pourquoi nous avons tout de même choisi de conserver cette configuration inattendue. Puis, nous avons dû aussi avoir un jeu de données assez conséquent pour son entrainement et son évaluation/validation. Nous avons donc séparé le dataset fournit, *Iris_extended*, avec un jeu d'entrainement à 80% et un de validation à 20% des instances. (cf. Table 3)

Classes	Jeu d'entrainement	Jeu de validation	Instances total
Setosa	313	87	400
Virginica	327	73	400
Versicolor	320	80	400
Total	960	240	1200

TABLE 1 – Table de répartissions des instances du jeu de données *Iris_extended* ordonnées par classes

2.1.2 Architecture et entrainement

Chaque architecture de modèles sont basés sur deux couchées de 16 et 8 neurones respectivement. Nous avons choisi deux type de fonction d'activation à tester pour la création de notre modèle :

1. **Relu** avec une taille de *batch* de 4, 1000 époques et un *learning rate* de 0.000000745. Ce *learning rate* très bas est expliqué par la formule donnée section 2.1.1.
2. **Tanh** avec une taille de *batch* de 4, 1000 époques et un *learning rate* de 0.0005.

2.2 Sélection d'instances

Afin d'interpréter la prédiction d'un réseau de neurones par des modèles locaux, nous devons d'abord choisir six instances issues de l'ensemble de validation : trois correctement classées, et trois incorrectement classées. Nous examinerons ensuite la contribution des attributs d'entrée à leur prédiction. Notre objectif est de sélectionner des instances à la fois représentatives mais aussi intéressantes à interpréter. Pour cela, nous avons appliqué les critères suivants :

- **Proximité de la frontière de décision :** les instances proches de la frontière de décision du modèle sont les plus susceptibles de changer de classe en réponse à de légères perturbations (voir Figure 1). Cela permet de générer un ensemble localement varié et donc utile à l'entraînement d'un modèle interprétable comme un arbre de décision ou une régression linéaire.
- **Niveau de confiance (incertitude) :** pour mesurer cette proximité, nous allons analysé les vecteurs de prédiction (probabilités softmax)

générés par notre réseau de neurones. Plus la probabilité maximale d'une prédiction est faible, plus le modèle est incertain. Nous allons donc sélectionné les instances pour lesquelles la sortie du modèle montre le plus faible score de confiance.

- **Diversité des cas :** nous avons choisi trois instances correctement classées (la prédiction du modèle correspond à la vérité terrain) et trois incorrectement classées (le modèle se trompe). Cela permet d'étudier les cas où le modèle réussit à prendre une bonne décision malgré une incertitude, ainsi que les cas où une mauvaise décision a été prise.

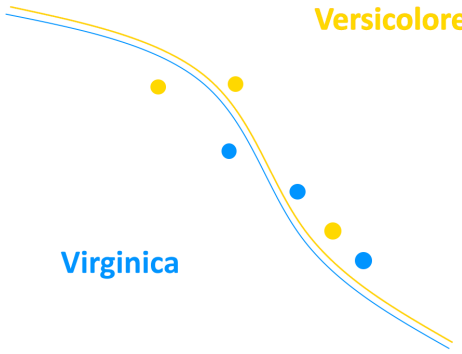


FIGURE 1 – Illustration de six instances proches de la frontière de décision

Notre fonction de sélection repose sur une légère adaptation du modèle pour extraire, pour chaque instance, à la fois la véritable étiquette, la prédiction du modèle et le vecteur de probabilités softmax. Nous trions ensuite les instances par valeur maximale du vecteur de sortie (du plus bas au plus haut), en séparant les bien classées des mal classées [Annexe A]. Nous avons également volontairement réduit légèrement les capacités de notre modèle afin d'obtenir les quelques erreurs de classification nécessaires.

2.3 Interprétation du modèle

Afin de mieux comprendre comment le réseau de neurones prend ses décisions, nous avons mis en place une approximation locale de son comportement autour de chaque instance sélectionnée. Cela nous permettra d'évaluer la contribution individuelle de chaque attribut à une prédiction spécifique.
Pour chaque instance sélectionnée à l'étape précédente, nous générons un

ensemble local composé de 250 instances perturbées. Chaque instance perturbée est obtenue en modifiant aléatoirement chaque attribut de l'instance originale, à hauteur de $\pm 10\%$ de sa valeur initiale. Ces perturbations sont appliquées de manière indépendante sur chaque attribut, en ajoutant une variation uniforme proportionnelle à la valeur de l'attribut (pour chaque valeur de l'instance : `val + uniform(-force * val, force * val)`). Ce processus est répété 250 fois pour constituer un ensemble local représentatif.

L'objectif est de simuler un voisinage de l'instance dans l'espace des attributs afin de capturer le comportement du réseau dans cette zone locale, comme l'illustre la Figure 2.

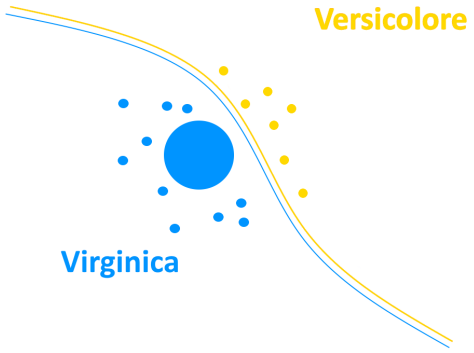


FIGURE 2 – Instances bruitées (petits points) au voisinage de l'instance sélectionnée (gros point)

Les 250 instances perturbées associées à une instance de référence sont ensuite passées dans le réseau de neurones initialement entraîné. Nous récupérons pour chacune la classe prédite, ce qui nous permet de constituer un jeu de données localement étiqueté, propre à cette instance.
Nous avons choisi d'utiliser des arbres de décision, de profondeur **4**, comme modèles interprétables pour approximer le comportement du réseau dans le voisinage local de chaque instance. Un arbre est entraîné individuellement pour chaque instance, en utilisant son jeu de perturbations. L'arbre de décision est mis en œuvre *from scratch*, en reprenant le code développé lors du TP 2. La profondeur maximale de l'arbre est fixée à 4, afin de conserver une bonne lisibilité et un pouvoir explicatif local raisonnable. Chaque arbre obtenu fournit une approximation locale de la fonction de décision du réseau de neurones, dans la zone de l'espace correspondant aux perturbations d'une

instance donnée.
Les attributs situés au sommet de l'arbre (ou impliqués dans les premières divisions) sont donc ceux qui ont le plus contribué à la décision du réseau dans ce voisinage.

2.4 Examen de la contribution des attributs

À chaque étape de l'algorithme de construction de l'arbre, l'attribut le plus décisif est calculé en fonction de son gain d'information. La valeur de contribution que nous attribuons à nos attributs est la division du gain d'information à l'étape de l'algorithme de construction ou l'attribut est choisi divisé par la hauteur. Cette approche se justifie par le fait que le gain est recalculé à chaque niveau, et un attribut situé en profondeur peut présenter un gain élevé, possiblement supérieur à celui d'un autre situé plus proche de la racine. Or, la contribution de celui proche de la racine est plus importante car elle implique un plus grand nombre de décisions en intervenant plus tôt dans le processus de tri des données. D'où nous faisons cette pondération.

3 Résultats

3.1 Capacités de prédiction du modèle

Nous avons donc tout d'abord utilisé le modèle basé sur la fonction d'activation **relu**. Dans un première temps nous avons tester l'impact de notre formule du *learning rate* sur l'apprentissage d'un même modèle. Aussi, afin de découvrir d'autres proprités de celle-ci, nous avons fait nos tests sur 10000 époques. Voici la figures 3 qui illustre l'entraînement sans notre formule, et la figure 4 qui illustre l'entraînement avec celle-ci.

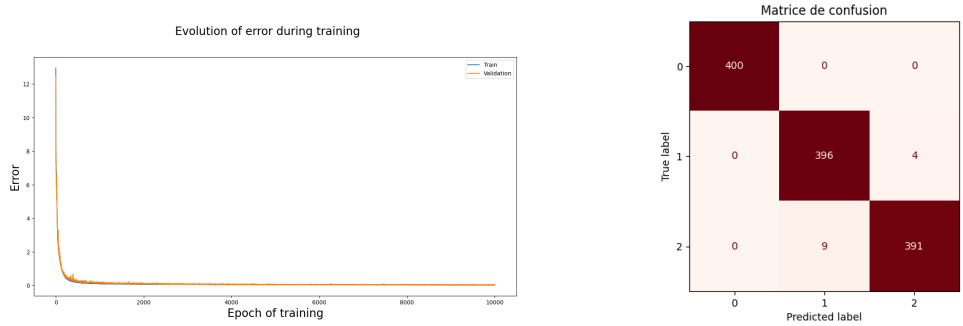


FIGURE 3 – Courbe d'apprentissage et matrice de confusion sur l'ensemble du jeu de données pour notre réseau de neurones **Relu** sans la fonction de *learning rate*

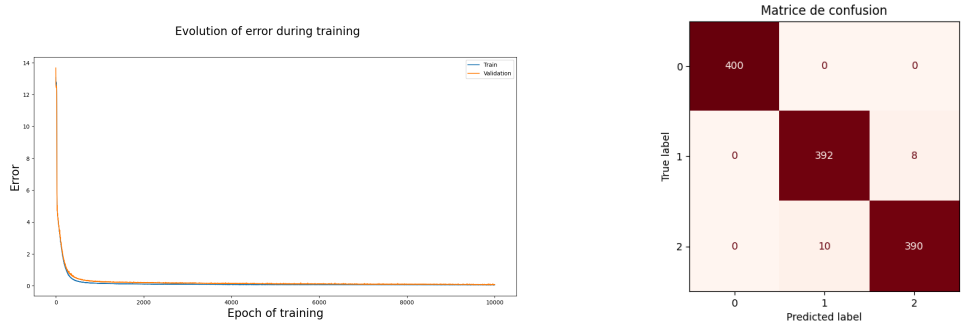


FIGURE 4 – Courbe d'apprentissage et matrice de confusion sur l'ensemble du jeu de données pour notre réseau de neurones **Relu** avec la fonction de *learning rate*

De plus, pour évaluer la performance de nos modèles, nous avons utilisé notre jeux de validation dans un premier temps, puis l'entièreté de notre dataset dans un second temps pour une meilleur visualisation des résultats, afin de construire des matrices de confusion.

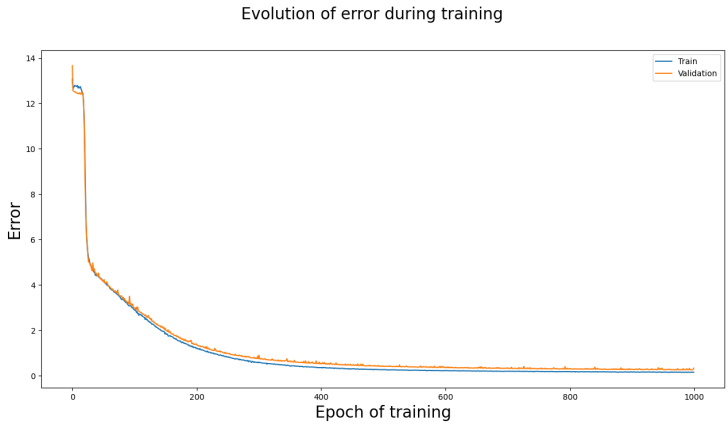


FIGURE 5 – Courbe d'apprentissage du modèle avec la fonction d'activation **Relu**

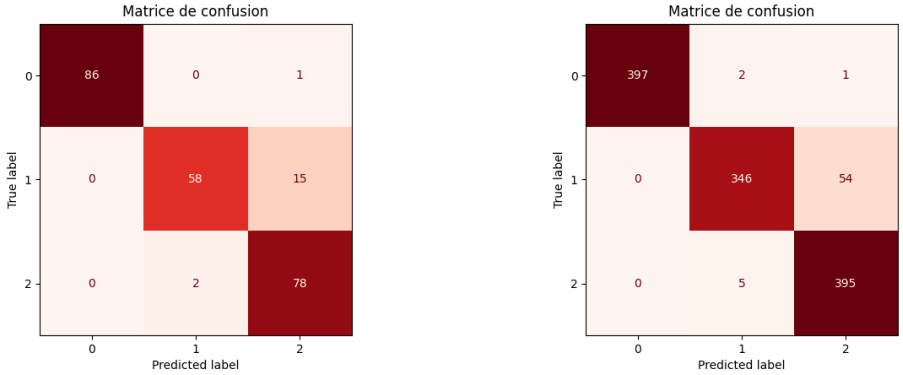


FIGURE 6 – Matrice de confusion avec le jeu de validation (à gauche) et global (à droite) du modèle avec la fonction d'activation **Relu**

La particularité de la fonction d'activation **Tanh** est qu'elle nécéssite la normalisation des données du dataset pour pouvoir correctement entrainé le modèle. Cela est dû à son domaine de définition qui se situe entre [-1 ;1].

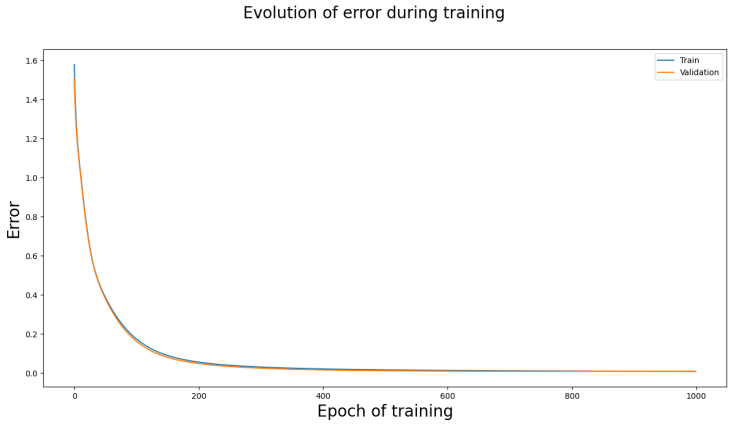


FIGURE 7 – Courbe d'apprentissage du modèle avec la fonction d'activation **Tanh**

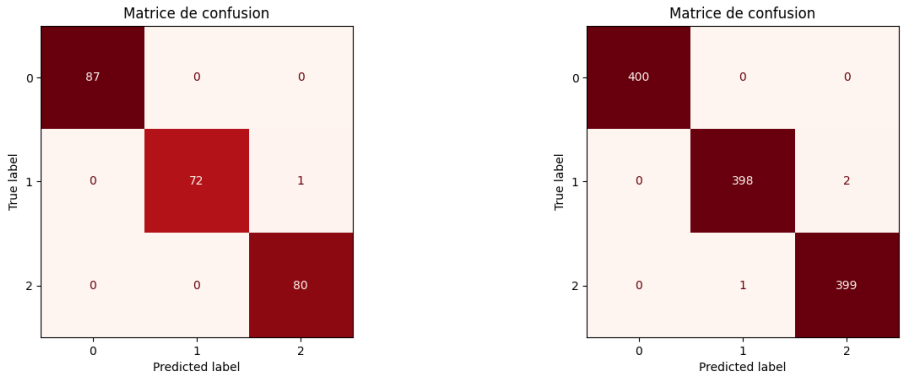


FIGURE 8 – Matrice de confusion avec le jeu de validation (à gauche) et global (à droite) du modèle avec la fonction d'activation **Tanh**

Grâce à ces matrices de confusion, nous avons pu construire un tableau reprenant les caractéristiques d'évalution de notre modèle.

Fonction d'activation	Accuracy	Précision	Recall	F1-score
Relu global	94.8%	95.3%	94.8%	94.8%
Relu validation	92.5%	92.9%	91.9%	91.9%
Tanh global	99.75%	99.7%	99.7%	99.7%
Tanh validation	99.6%	98.7%	99.1%	98.9%

TABLE 2 – Table de répartissions des instances du jeu de données *Iris_extended* ordonnées par classes

Pour la suite des expérimentation nous avons pris le modèle basé sur la fonction tanh pour des raisons expliciter dans la section 4 correspondant à la discussion.

3.2 Instances choisies

Comme explicité dans la section 2.2, nous avons selectionné nos 6 instances, correctement et mal prédites, à l'aide de la fonction A.

Classes	Good-1	Good-2	Good-3	Bad-1	Bad-2	Bad-3
Setosa	0.005	0.006	0.004	0.005	0.003	0.006
Virginica	0.406	0.646	0.320	0.506	0.390	0.345
Versicolor	0.588	0.347	0.674	0.488	0.605	0.647

TABLE 3 – Vecteurs de probabilité donnés par le réseau de neurones pour les instances correctement prédites (Good) et mal prédites (Bad) sélectionnées

3.3 Comportement local du modèle

Dans le tableau 4 les modéles locales 1 à 3 ont été entraînés respectivement sur les jeux de données créés par le voisinages des trois instances choisies, difficiles à prédire mais prédit correctement par le réseau de neurones.

Tandis que les modèles 3 à 6 sont ceux entraînés sur les jeux de données crées par le voisinage des trois instances choisies, difficile à prédire et mal prédits. L'alignement des décisions est la proportion de prédictions identiques par le modèle local correspondant et le réseau de neurones

Modèle local	**Alignement des décisions (%)**
Modèle local 1	63.20
Modèle local 2	62.40
Modèle local 3	67.60
Modèle local 4	57.60
Modèle local 5	63.20
Modèle local 6	60.40

TABLE 4 – Comparaison des prédictions des modèles locaux entraînés par les instances choisies et perturbées à 10% avec ceux du réseau de neurones

Le tableau 5 est similaire à celui qui précède, au lieu de 10 % de perturbation, 20 % de perturbation est utilisé.

Modèle local	**Alignement des décisions (%)**
Modèle local 1	58.00
Modèle local 2	56.40
Modèle local 3	62.00
Modèle local 4	57.60
Modèle local 5	60.80
Modèle local 6	57.60

TABLE 5 – Comparaison des prédictions des modèles locaux entraînés par les instances choisies et perturbées à 10% avec ceux du réseau de neurones

D'aprés le tableau 4 les alignements des décisions sont toutes au dessus de 57,60% et sont en moyenne est de 62.4% . Tandis que d'aprés le tableau 5 les alignement des décisions sont toutes au dessus de 56,40% et sont en moyenne de 58.73%.

3.4 Contribution des attributs

Dans le figure 9, nous pouvons voir les contributions des attributs de chaque modèle avec une perturbation de 10% sur les instances choisies. En-

suite, dans le figure 10 nous pouvons voir les contributions des attributs, cette fois-ci avec une perturbation de 20% .

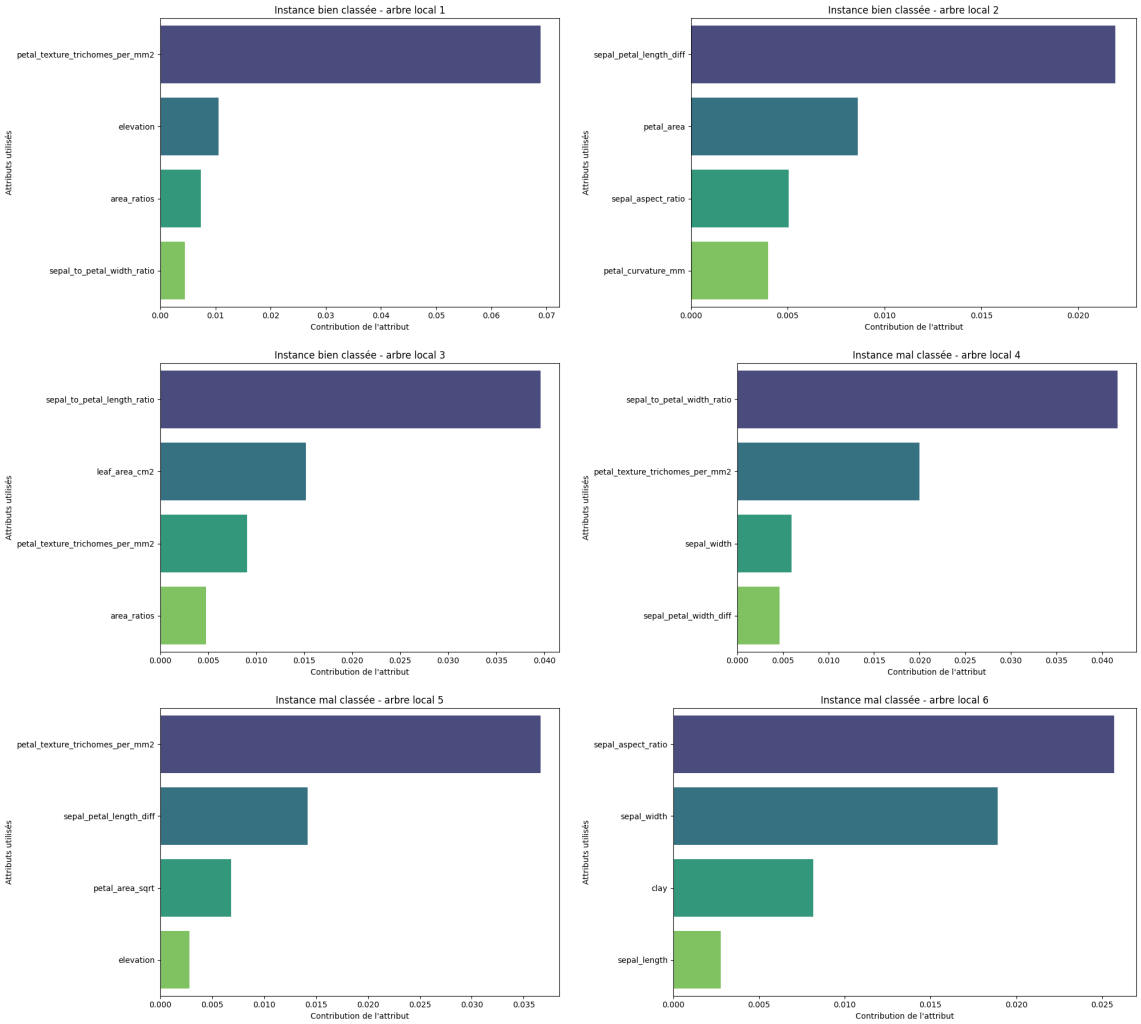


FIGURE 9 – Contribution des attributs pour les modèles locales à partir des instances choisies et perturbées de 10%

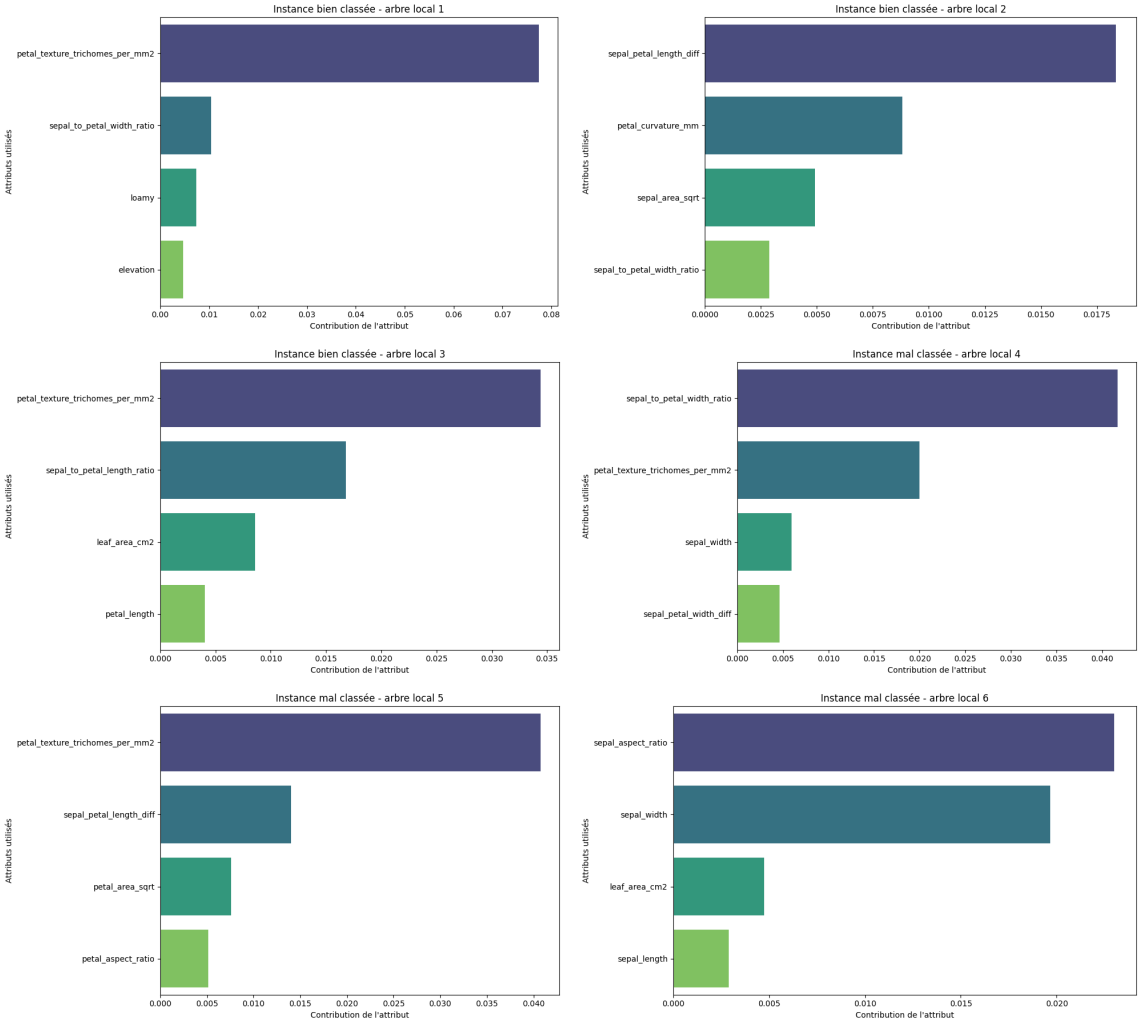


FIGURE 10 – Contribution des attributs pour les modèles locales à partir des instances choisies et perturbées de 20%

Ces derniers montrent que les attributs les plus importants restent les mêmes quand la perturbation est augmentée, de même, leur degré de contribution reste presque identique. Nous voyons que pour les arbres faites par les instances mal classées et les instances bien classées, sauf le dernier modèle. L'attribut le plus important, le premier de l'arbre est l'attribut prédominant, jouant une rôle beaucoup plus important que les autres. Pour l'arbre local 6, nous pouvons voir que le deuxième est presque aussi contribuant que le

premier.

4 Discussion

Nous avons eut plusieurs résultats les uns à la suite des autres et voici une possible analyse de ceux-ci dans l'ordre chronologique :

— **La fonction de learning rate inversé :** Dans un premier temps, nous avions supposé que celle-ci nous permettait d'avoir de meilleur résulats que sans et au vu de nos nombreuses expèriences sans overfitting apparent, nous avions aussi supposé que celle-ci pourrait limité son apparition. Cependant, comme observer dans notre expèrience de **10000 époques**, les résulats sont toujours biens meilleurs lorsque l'on ne l'utilise pas. L'overfitting apparaît même plus tôt avec notre formule. Nous l'avons donc recyclé pour notre modèle basé sur **Tanh** pour limité légèrement son accuracy. Cela nous as permit de géré uniquement et parfaitement 3 erreurs de prédictions. Nous avons aussi étonnament remarqué que c'est la classe des **Versicolor** qui était la plus complexe à prédire pour ces modèles.

— **Le meilleur modèle :** Nous avions donc créé deux modèle basé sur deux fonctions d'activation différentes, **Relu** et **Tanh**. Comme observé à l'aide du tableau, nous avons remarqué que le modèle de **Tanh** était bien meilleur que celui du **Relu**, avec une différence d'envirion 4% d'accuracy. Nous l'avons donc logiquement choisie pour la suite de nos travaux malgré la nécéssité de réduire ses capacités pour le bien de nos expérimentations. Comme observé, la classe des **Virginica** est la classe la plus difficile a prédire pour tout nos modèle. De plus, avec le très faible décalage entre la précision et le recall, de l'ordre de 0.4%, nous pouvons aussi dire qu'aucun overfitting n'as été observé.

— **Instances choisies :** Les instances selectionnés par nos fonctions sont donc celles où le modèle à le plus de doute. Cependant, nous pouvons remarque que le modèle ne vas jamais douter sur les 3 classes à la fois. Dans notre cas, le modèle n'as jamais de doute sur la classe des **Setosa** mais plutôt entre celle des **Virginica** et des **Versicolor**. Aussi, nous pouvons remarquer que le modèle est bien plus certain, en moyenne, pour la classe des **Versicolor** que pour les **Virginica**.

— **Modèles locaux :** Nos 6 arbres de décisions basé sur des jeux d'ins-

tances perturbées ont été plus ou moins pertinant pour approximée notre réseau de neurones. Nous avons remarqué que pour 10% de perturbation la moyenne se situe à 62.4% et que pour 20% de perturbation 56.40%. Nous pouvons en déduire que plus nos données sont éparpillé, plus notre arbre à des difficultés à correctement prédire. Nos arbres sont donc très moyen pour reproduire notre modèle. Cependant, nous pouvons tout de même remarqué que si les jeux de donnés créé à partir d'instances bien prédites font de meilleurs score que ceux mal prédites pour 10% de perturbation, elles sont tout de même plus stable avec une réduction moyenne de 3% pour 20% de perturbation, alors que les bien prédites annonce un décalage d'environ 5%. Cela s'applique surtout à notre **modèle local 4** qui reste parfaitement stable avec 10% ou 20% de perturbation. Pour améliorer ces scores, nous pouvons émettre l'hypothèse qu'un arbre avec une profondeur **supérieur à 4** pourrait mieux interprété localement notre modèle.

— **Contribution des attributs :** Nous avons pu remarqué que aucun attributs n'était réelement similairement impactant dans la prédiction final des arbres. Chaque modèles locaux ont donc une contribution différente de leur attributs. De plus, au vu de l'éfficacité de ces modèles pour interprété localement notre réseau de neurone, nous pouvons supposé que ces contributions ne sont pas réelement représentatif de notre réseau car les prédictions de ces derniers ne s'alignent pas assez bien.

5 Conclusion

Ainsi, au cours de nos expériences, tout d'abord, nous avons le modele avec la fonction d'activation, tanh, était le plus adapté pour nos données et nécessitait une normalisation de celle-ci. Ensuite, nous avons choisis des instances, où le modele doutait beaucoup afin de diversifié nos jeux de données perturbées. Puis, la création des modeles locales a partir de ces jeux de données, nous a montré que les arbres que nous avons conçu ne sont pas suffisament pertinent pour reproduire le comportement local exacte de notre reseau de neurones. Tout de meme, l'interpration de ces arbres nous ont indiqués les attributs plus prédominants d'aprés notre méthode d'évaluation
.

Annexes

A Sélection d'instances

```
def selected_instance(nn,X_test,y_test):
    predictions = nn.predict(X_test.transpose())
    association_predictions = list(zip(predictions,y_test,range(len(y_test))))
    predictions_sorted = sorted(association_predictions, key=lambda v : max(v[0]))

    bad_set = []
    good_set = []
    ind = 0

    while len(bad_set) < 3 or len(good_set) < 3 :

        if ind >= len(predictions_sorted) :
            raise Exception("Instances not found !")

        pred_vector, true_label, idx = predictions_sorted[ind]
        rp = round_prediction(pred_vector)

        if np.array_equal(rp,true_label) and len(good_set) < 3 :
            print("Good : ",pred_vector)
            good_set.append(idx)

        elif len(bad_set) < 3 :
            print("Bad : ",pred_vector)
            bad_set.append(idx)

        ind += 1

    return good_set,bad_set
```

Fonction permettant de sélctionner nos 6 instances